CS 2840: Optimal Transport and Machine Learning

Final Project – Fall 2025

Are OT-Aligned Flows What You Need?

Name(s): J. Benarroch, M. Letey, P. Nair, P. Nielsen, Y. Zimmerman

Submission Date: 11/30/2025

Abstract

We study how the geometry of continuous-time generative models affects both sampling efficiency and downstream control. Within a unified probability-path view, we compare Diffusion Models, Flow Matching (FM), OT-inspired FM with Gaussian (McCann) interpolants, OT-Conditional Flow Matching (OT-CFM), and Rectified Flow (RF), under matched architectures and training pipelines. On CIFAR-10, we evaluate FID, path straightness, and kinetic energy, and find that although OT-CFM learns the straightest, lowest-energy flows, it does not outperform the simpler straight-line RF paths in sample quality. We then replace Diffusion Policys denoising process with CFM-based ODE policies on a cube-stacking task. We find that CFM-based policies achieve high one-shot success rates and robust out-of-distribution generalization, while Diffusion Policy collapses under single-step inference. Furthermore, we find that CFM-based policies yield straighter denoising trajectories with lower energies.

1 Introduction

Generative modeling in continuous time has undergone a rapid transformation, driven by a unifying view in which generation is realized as a probability path $(\mu_t)_{t\in[0,1]}$ pushing a simple base distribution toward the data distribution through stochastic or deterministic dynamics [1]. Classic score-based diffusion models [2–4] instantiate this paradigm via a forward stochastic differential equation (SDE) that gradually destroys structure, paired with a learned reverse-time SDE that reconstructs samples through estimated score fields. The probability-flow reformulation [4] demonstrates that these same marginals may also be realized by a deterministic ordinary differential equation (ODE), revealing diffusion models as one instance among a broader family of flow-based generative processes.

Flow Matching (FM) [5] makes this ODE perspective explicit. Rather than learning scores of noisy intermediate distributions, FM directly regresses a neural vector field toward the velocity of a chosen reference interpolation between source and data distributions. This yields a simulation-free training objective and a generator defined purely as an ODE flow. Subsequent work has highlighted that diffusion models, FM, and their variants can be understood through the continuity equation and a choice of probability path [1], suggesting that improvements in sample quality or efficiency may hinge on the geometry of the chosen path rather than on the specific algorithmic formalism.

A particularly influential geometric lens is provided by optimal transport (OT). In the Benamou-Brenier dynamic formulation of OT [6, 7], the W_2 distance corresponds to the minimal kinetic-

[0]Code available at https://github.com/mletey/CS2840-ot-aligned-flows for image-based experiments and https://github.com/JackBJ23/cfm-policies/tree/main for robot-based experiments.

energy flow between distributions. This perspective connects directly to the "straightness" and energetic efficiency of generative trajectories. Rectified Flow (RF) [8, 9] operationalizes this idea by iteratively adjusting the learned ODE so that its characteristics align more closely with displacement interpolations. In practice, this leads to shorter and smoother trajectories that require fewer integration steps. OT-aligned variants of FM, such as OT-CFM (Conditional Flow Matching) [10] and Multisample FM [11], further exploit this geometry by constructing conditional objectives or minibatch couplings that approximate OT geodesics and encourage low-energy transport.

Despite these converging theoretical insights, it remains unclear how much of the empirical improvement in speed or sample quality arises from OT alignment itself, as opposed to architectural choices, noise schedules, or optimization effects; systematic empirical evidence disentangling these factors remains limited.

In this work, we conduct a controlled comparison of four continuous-time generative modeling paradigms: (1) Diffusion models; (2) Flow Matching with OT-inspired conditional paths (FM), in which each data point is paired with a straight-line Gaussian displacement (a McCann interpolant [6, 12]); (3) OT-consistent Flow Matching (OT-CFM), which augments this formulation with minibatch OT couplings to approximate the globally optimal W_2 transport between the base and data distributions; and (4) Rectified Flow, corresponding to zero-variance, straight-line conditional paths within the CFM framework.

Under matched architectures, datasets, and training pipelines, we evaluate these models on two different settings. First, we evaluate them on CIFAR-10 [13] using both standard generative metrics and geometry-aware diagnostics motivated by OT theory, including trajectory straightness and Benamou-Brenier kinetic energy.

Next, we examine whether these advantages persist in control settings by applying the same models to policy generation in robotics. In particular, we consider a stack-the-cube task using a Panda robot arm in the SAPIEN simulator [14] and study generalization under an obstacle inserted between the cubes. We compare our flow-based policies against Diffusion Policy [15], the state-of-the-art imitation-learning approach.

Finally, we note that our experiments, combining the CFM framework with the ManiSkill and SAPIEN environments for robotics and the diffusion policy framework, are to the best of our knowledge novel work. Furthermore, our results regarding straightness and energy convergences and discrepancies between CFM-based policies and Diffusion Policy are also novel. The 1-shot action generation and generalization enabled by CFM-based policies in our tasks –capabilities not exhibited by Diffusion Policy – are, to the best of our knowledge, also new results.

2 Methods

Continuous Normalizing Flows We work on a data space $\mathbb{R}^d$ with points x. Given a time-dependent vector field $v_t : \mathbb{R}^d \to \mathbb{R}^d$, we define its associated *flow* $\phi_t : \mathbb{R}^d \to \mathbb{R}^d$ as the solution of the ODE

$$\frac{d}{dt}\,\phi_t(x) = v_t(\phi_t(x)), \qquad \phi_0(x) = x. \tag{1}$$

Under mild regularity assumptions, each ϕ_t is a diffeomorphism, and it transports an initial ("prior") density p_0 along the flow via the pushforward

$$p_t = (\phi_t)_{\#} p_0, \qquad (\phi_t)_{\#} p_0(x) = p_0\big(\phi_t^{-1}(x)\big)\, \det\left(\frac{\partial \phi_t^{-1}}{\partial x}(x)\right). \tag{2}$$

In words, v_t generates the *probability path* $(p_t)_{t\in[0,1]}$ if the solution of (1) pushes p_0 forward to p_t as in (2). Equivalently, (p_t, v_t) satisfy the continuity equation

$$\partial_t p_t(x) + \nabla \cdot \big(p_t(x)\, v_t(x)\big) = 0, \tag{3}$$

which encodes conservation of probability mass along the flow.

Continuous Normalizing Flows (CNFs) [16] parametrize v_t by a neural network $v_t(x;\theta)$ with parameters θ, thereby inducing a flexible family of flows ϕ_t and corresponding density paths p_t. CNFs can in principle realize a very broad class of probability paths, including those arising from diffusion models via their probability-flow ODEs [4]. However, standard maximum-likelihood training requires estimating log-density changes along ODE trajectories (e.g., via instantaneous change-of-variables), which entails repeated numerical ODE solves and divergence estimates [17]. This makes naive CNF training computationally expensive. Flow Matching (FM) [5] was proposed precisely to bypass this bottleneck by turning CNF training into a simple regression problem on vector fields.

Flow Matching Assume we are given an unknown data distribution $q(x_1)$ from which we can sample but whose density is not known, and a *target* probability path p_t satisfying

$$p_0 = p \quad (\text{e.g. } \mathcal{N}(0, I)), \qquad p_1 \approx q.$$

Suppose further that we know a vector field u_t that generates p_t. The idea of Flow Matching is then to train the CNF vector field $v_t(\cdot;\theta)$ to approximate u_t directly via a mean-squared error objective:

$$\mathcal{L}_{\mathrm{FM}}(\theta) = \mathbb{E}_{t\sim\mathcal{U}[0,1],\, x\sim p_t} \big\| v_t(x;\theta) - u_t(x) \big\|^2. \tag{4}$$

Conceptually, (4) is extremely appealing as it turns CNF training into a simulation-free regression problem. In practice, however, neither p_t nor u_t is tractable, since both require marginalizing over the data distribution. FM resolves this via conditional paths defined per data point, which yield an equivalent but tractable regression objective.

From Conditional Paths to Marginal Dynamics Let $x_1 \sim q$ be a data sample. For each x_1, define a *conditional* path $p_t(x \mid x_1)$ such that

$$p_0(x \mid x_1) = p(x) \quad (\text{the same simple prior for all } x_1),$$
$$p_1(x \mid x_1) \approx \delta_{x_1}(x) \quad (\text{e.g. a narrow Gaussian centered at } x_1).$$

Aggregating these conditional paths over the data distribution induces a *marginal* probability path

$$p_t(x) = \int p_t(x \mid x_1)\, q(x_1)\, dx_1. \tag{5}$$

Then $p_0 = p$ and p_1 is a mixture that approximates q:

$$p_1(x) = \int p_1(x \mid x_1)\, q(x_1)\, dx_1 \approx q(x).$$

Similarly, for each x_1 we assume a conditional vector field $u_t(\cdot \mid x_1)$ that generates the conditional path. We then define a *marginal* vector field u_t by averaging the conditional fields:

$$u_t(x) = \int u_t(x \mid x_1)\, \frac{p_t(x \mid x_1)\, q(x_1)}{p_t(x)}\, dx_1, \qquad p_t(x) > 0. \tag{6}$$

This weighted average may not look obviously related to the true velocity field of the marginal path p_t, but the key structural result of Flow Matching is that it is in fact the *correct* marginal generator.

Theorem 1 ([5], Thm. 1)**.** *Suppose that, for each x_1, the vector field $u_t(\cdot \mid x_1)$ generates the conditional path $p_t(\cdot \mid x_1)$, and let p_t and u_t be defined by (5) and (6). Then (p_t, u_t) satisfy the continuity equation (3), i.e., u_t generates the marginal probability path p_t.*

Conditional Flow Matching The marginal FM objective (4) is still intractable, since evaluating p_t or u_t requires marginalizing over q. Conditional Flow Matching (CFM) replaces the marginal regression (4) with a conditional regression over the paths $p_t(\cdot \mid x_1)$:

$$\mathcal{L}_{\mathrm{CFM}}(\theta) = \mathbb{E}_{t\sim\mathcal{U}[0,1],\, x_1\sim q,\, x\sim p_t(\cdot|x_1)} \big\| v_t(x;\theta) - u_t(x \mid x_1) \big\|^2. \tag{7}$$

This objective is simulation-free: as long as we can sample $x_1 \sim q$, then $x \sim p_t(\cdot \mid x_1)$, and evaluate the conditional target field $u_t(x \mid x_1)$ in closed form, we can estimate (7) by Monte Carlo without solving any ODEs or computing any marginal densities. It is equivalent to the marginal FM loss:

Theorem 2 ([5], Thm. 2)**.** *Assume $p_t(x) > 0$ for all $x \in \mathbb{R}^d$ and $t \in [0, 1]$. Then the Flow Matching and Conditional Flow Matching losses differ only by a constant independent of θ.*

Thus, optimizing the conditional objective (7) is exactly equivalent (in expectation) to optimizing the ideal marginal objective (4).

Gaussian Conditional Paths [5] propose conditional paths of the form

$$p_t(x \mid x_1) = \mathcal{N}\big(x \,|\, \mu_t(x_1),\, \sigma_t(x_1)^2 I\big), \tag{8}$$

where the time-dependent mean $\mu_t : \mathbb{R}^d \to \mathbb{R}^d$ and scalar standard deviation $\sigma_t : \mathbb{R}^d \to \mathbb{R}_{>0}$ satisfy the boundary conditions

$$\mu_0(x_1) = 0, \quad \sigma_0(x_1) = 1, \qquad \mu_1(x_1) = x_1, \quad \sigma_1(x_1) = \sigma_{\min},$$

for a small $\sigma_{\min} > 0$.

There are infinitely many vector fields that generate a given probability path (one can always add divergence-free components that preserve p_t). To obtain a simple and canonical choice, [5] focus on the affine flow

$$\psi_t(x) = \sigma_t(x_1)\, x + \mu_t(x_1), \tag{9}$$

viewed as a map from a standard Gaussian $x \sim \mathcal{N}(0, I)$ to the conditional distribution (8). By the usual change-of-variables formula, ψ_t pushes $p_0(\cdot \mid x_1) = p(\cdot)$ forward to $p_t(\cdot \mid x_1)$. The corresponding conditional vector field $u_t(\cdot \mid x_1)$ is defined by differentiating the flow:

$$\frac{d}{dt}\psi_t(x) = u_t(\psi_t(x) \mid x_1). \tag{10}$$

Substituting this into the CFM objective yields the practically used form

$$\mathcal{L}_{\mathrm{CFM}}(\theta) = \mathbb{E}_{t\sim\mathcal{U}[0,1],\, x_1\sim q,\, x_0\sim p}\left\|v_t(\psi_t(x_0);\theta) - \tfrac{d}{dt}\psi_t(x_0)\right\|^2, \tag{11}$$

which only requires sampling (t, x_1, x_0) and evaluating the affine map ψ_t and its time derivative. Because ψ_t is affine in x, the associated vector field $u_t(\cdot \mid x_1)$ admits a simple closed form:

Theorem 3 ([5], Thm. 3)**.** *Let $p_t(x \mid x_1)$ be a Gaussian conditional path of the form (8), and let ψ_t be the affine flow (9). Then the unique vector field $u_t(\cdot \mid x_1)$ satisfying (10) is*

$$u_t(x \mid x_1) = \frac{\sigma_t'(x_1)}{\sigma_t(x_1)}(x - \mu_t(x_1)) + \mu_t'(x_1), \tag{12}$$

where primes denote time derivatives. Moreover, $u_t(\cdot \mid x_1)$ generates the Gaussian path $p_t(\cdot \mid x_1)$.

Once μ_t and σ_t are specified, training reduces to approximating this analytically defined vector field with a neural CNF $v_t(\cdot;\theta)$ via the simulation-free objective (11). In the following sections, we instantiate particular choices of (μ_t, σ_t) that recover classical diffusion paths as well as optimal-transport-aligned flows, and we compare their empirical and geometric properties in a controlled setting.

Rectified Flows (RF) A particularly simple conditional path is obtained by the straight-line interpolation

$$x_t = (1-t)x_0 + tx_1, \qquad \sigma_t = 0,$$

so that $\mu_t = (1-t)x_0 + tx_1$. By Theorem 3 of [5], the associated vector field is constant:

$$u_t(x \mid x_0, x_1) = x_1 - x_0.$$

This produces a constant-speed straight-line transport and behaves as a one-step model, since a single Euler update maps x_0 to x_1. This version of RF corresponds to the $\sigma_t = 0$ special case of Independent CFM (I-CFM) [10].

Flow Matching with OT Let $p_0 = \mathcal{N}(0, I)$ and $p_1 = \mathcal{N}(x_1, \sigma_{\min}^2 I)$. The W_2 displacement interpolation between p_0 and p_1 (the McCann interpolant; see [12]) is given by

$$p_t = \left[(1-t)\,\mathrm{id} + t\,\psi\right]_\sharp p_0,$$

where ψ is the OT map from p_0 to p_1. For Gaussian-to-Gaussian transport, the OT map has a closed form, and the displacement interpolation simplifies to

$$\psi_t(x) = \big(1 - (1-\sigma_{\min})t\big)x + tx_1. \tag{13}$$

Matching this OT interpolant with the affine Gaussian CFM parameterization yields

$$\mu_t(x_1) = t\, x_1, \qquad \sigma_t(x_1) = 1 - (1 - \sigma_{\min})t,$$

so that $\psi_t(x) = \sigma_t(x_1)x + \mu_t(x_1)$. By Theorem 3 of [5], the associated conditional vector field is

$$u_t = \frac{x_1 - (1 - \sigma_{\min})x_t}{1 - (1 - \sigma_{\min})t},$$

corresponding exactly to the OT displacement (McCann) interpolation between p_0 and p_1.

OT-Consistent Conditional Flow Matching (OT-CFM) Standard CFM samples endpoint pairs (x_0, x_1) independently from $q_0 \otimes q_1$, so that each conditional path is defined with respect to an unstructured pairing. OT-CFM replaces this with a *globally optimal* coupling: for each minibatch of base samples $\{x_i^0\}$ and data samples $\{x_j^1\}$, it computes the W_2 OT plan

$$\Pi^\star = \arg\min_{\Pi \in \mathcal{U}(q_0, q_1)} \sum_{i,j} \Pi_{ij}\, \|x_i^0 - x_j^1\|^2,$$

and draws endpoint pairs (x_0, x_1) jointly according to this minibatch OT plan. Once paired, OT-CFM employs exactly the same straight-line conditional paths used in RF/I-CFM,

$$x_t = (1 - t)x_0 + t x_1 + \sigma\, \varepsilon, \qquad u_t(x \mid x_0, x_1) = x_1 - x_0,$$

so the sole modification relative to standard CFM lies in the OT-consistent choice of endpoints. Proposition 3.4 of [10] shows that, as the variance $\sigma^2 \to 0$, the resulting marginal path p_t and velocity field u_t converge to the Benamou-Brenier dynamic OT solution between q_0 and q_1.

Because exact OT is computationally prohibitive for large datasets, minibatch OT is used as a practical approximation. A caveat is that minibatch OT is computed on raw state vectors, ignoring conditioning variables such as temporal or task context; in conditional or sequential domains, this may yield semantically misaligned pairings. Nevertheless, we include OT-CFM to assess how globally OT-aligned couplings influence flow-matching dynamics in both image generation and control.

Diffusion Models Diffusion models [2–4] are defined by a forward noising SDE

$$dX_t = f_t(X_t)\, dt + g_t\, dW_t, \tag{14}$$

which evolves $X_0 \sim q$ toward a tractable noise distribution (typically $\mathcal{N}(0, I)$). For common choices such as the VP and VE SDEs, the marginals $p_t(x \mid x_0)$ are Gaussian with known mean and variance. In particular, the VP SDE yields the conditional law

$$X_t = \alpha_t x_0 + \sigma_t \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I),$$

so that diffusion models fit exactly into the Gaussian CFM parameterization with

$$\mu_t(x_0) = \alpha_t x_0, \qquad \sigma_t = \sigma_t,$$

and analogous expressions hold for other diffusion families. The generative dynamics use the reverse-time SDE

$$d\tilde{X}_t = \left(f_t(\tilde{X}_t) - g_t^2 \nabla_x \log p_t(\tilde{X}_t)\right) dt + g_t\, d\bar{W}_t, \tag{15}$$

where the score $\nabla_x \log p_t$ is approximated by a neural network $s_\theta(x, t)$ trained via denoising score matching:

$$\min_\theta \ \mathbb{E}_{t,x_0,x_t} \left\| s_\theta(x_t, t) - \nabla_x \log p_t(x_t \mid x_0) \right\|^2. \tag{16}$$

We follow the DDPM parameterization [3] and train the model to predict the Gaussian noise in

$$x_t = \sqrt{\bar{\alpha}t} x_0 + \sqrt{1 - \bar{\alpha}t}\varepsilon.$$

For these Gaussian forward marginals, noise prediction is equivalent to denoising score matching up to a known time-dependent scaling. At sampling time we therefore use the standard discrete DDPM reverse updates, and optionally the deterministic DDIM update rule [3] for generating noise-free trajectories.

3 Experiments & Results

3.1 Image Generation

All deterministic image generation models are sampled by integrating their respective ODEs using the adaptive Dopri5/RK45 [18] with tolerance 10^{-5}. We report the resulting number of NFE as the sampling cost. The stochastic diffusion baseline is sampled with EulerMaruyama using its standard 1000-step schedule. As shown in Table 1, OT-aligned methods achieve strong generative performance while requiring only 190 NFEs. This indicates that the learned transport fields are relatively smooth and easy for the adaptive solver to integrate. In contrast, the diffusion probability-flow ODE attains a lower NFE but produces poor samples, showing that low solver effort alone does not imply a well-structured flow. Overall, OT-aligned flows deliver a favorable balance between sample quality and computational efficiency.

Table 1: Comparison of the different models at the final training checkpoint.

Model	FID ↓	Straightness ↑	Kinetic Energy ↓	NFE
OT-CFM	9.04	0.9847	5.83e+03	188.0
RF	8.76	0.9779	5.87e+03	194.0
FM w/ OT	8.78	0.9776	5.87e+03	194.0
Diffusion (stoch.)	37.46	/	/	1000.0
Diffusion Flow	311.24	0.9468	7.10e+01	99.0

The comparative results on CIFAR-10, as presented in Figure 1, demonstrate that explicitly enforcing OT alignment through minibatch coupling does not yield a significant empirical advantage in generated sample quality (FID) over simpler flow-matching techniques. Specifically, the RF and FM model with Gaussian conditional paths achieved the best FIDs (8.76 and 8.78, respectively), showing no meaningful difference compared to the OT-CFM model (FID 9.04). Interestingly, OT-CFM did achieve the best geometric metrics, resulting in the straightest trajectories and lowest kinetic energy, affirming its success in finding a path mathematically closer to the W_2 geodesic. This could suggest that the theoretically ideal geometric flow path is not the sole, or even primary, determinant of high-fidelity image generation, and that the robust, straight-line conditional paths

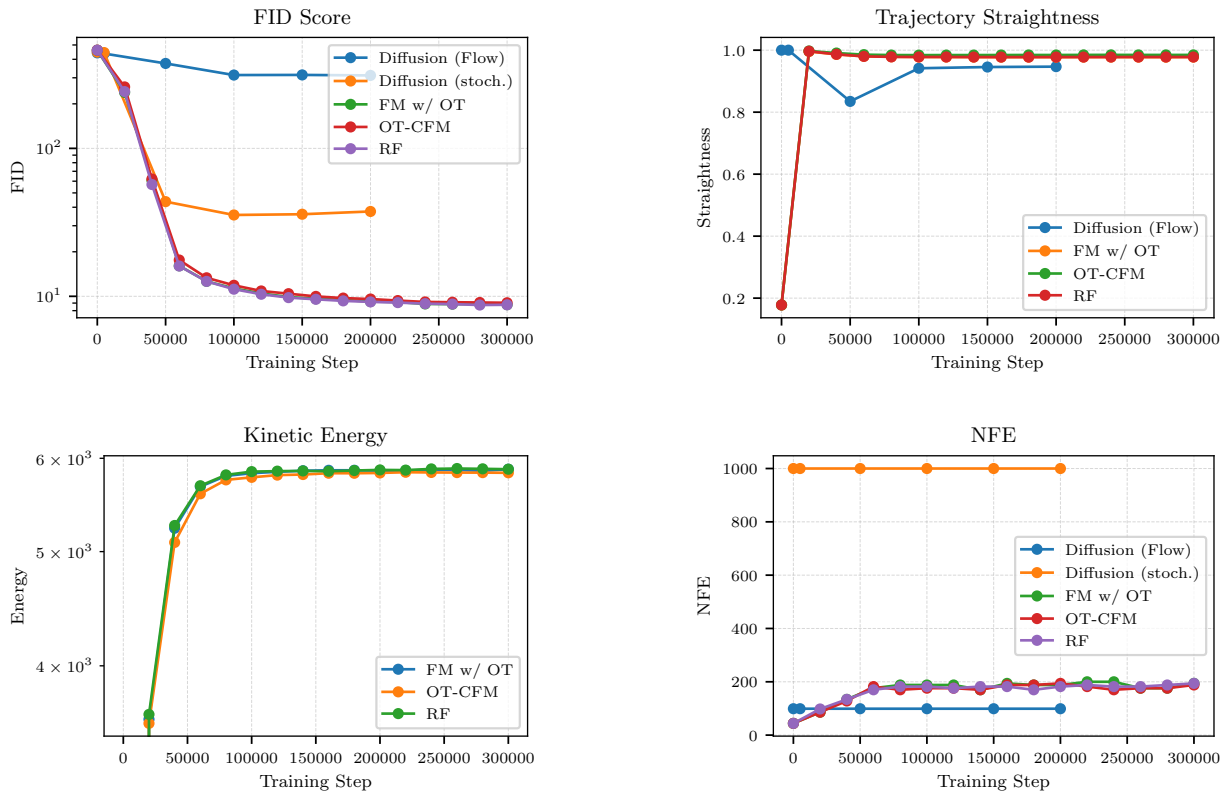

Figure 1: Training metrics over training time. FID (lower is better), Straightness (higher is straighter), Kinetic Energy (lower is better transport cost), and NFE (inference cost).

enforced by RF are sufficient for achieving SOTA performance in this setting while maintaining high sampling efficiency (low NFE).

We hypothesize that the marginal performance of the OT-aligned model is fundamentally limited by the necessary use of small-batch OT as a proxy for the true global W_2 geodesic. Our batch size of 128 is likely too small to reliably capture semantically meaningful, globally optimal couplings across the high-dimensional image manifold of CIFAR-10; this sentiment was echoed by Marco Cuturi during his lecture. This limited sample size introduces noise and suboptimal pairings, effectively leading to a noisy gradient signal. The noise introduced by this sub-optimal minibatch coupling appears to outweigh the benefits of enforcing global OT consistency, causing the OT-CFM variant to underperform our other proposed methods in terms of FID. Ultimately, our results here indicate that, at least for this setup, the inherent computational challenges of exact OT render its practical application less effective than simpler flow matching methods.

3.2 Robot Action Generation

We compare Diffusion Policy (DP) [15] with three CFM-based policies: FM, RF, and OT-CFM. While standard FM samples timesteps uniformly $\tau \sim \mathcal{U}(0, 1)$, the recently developed Vision-Language-Action foundation model π_0 [19] uses a Beta distribution biased toward small τ, emphasizing low timesteps (high noise levels). The distribution is given by $p(\tau) = \text{Beta}((s - \tau)/s; 1.5, 1)$, see Figure 9

for a visualization. We adopt this improved sampler for both FM and RF, and following [19] we use $s = 0.999$, yielding FM-π_0 and RF-π_0.

Stack-Cube All models are evaluated on a cube-stacking task designed in the ManiSkill simulator [20]: a Panda arm must pick the red cube and place it on the green one. We train all methods on the same 200 expert demonstrations generated by a motion planner we designed, using identical architectures (a 4.5M-parameter Conditional-UNet), learning schedules, seeds, and 100,000 training iterations with batch size of 512. Success rates are measured over 100 evaluation episodes every 5,000 steps, see Figure 10. All models use 100 integration steps during training. We can see that training performance is similar across models, with the FM and RF models overperforming in early training and the π_0 models slightly below in early training. OT-CFM, however, fails completely, since its OT plan matches raw state–action vectors without considering conditioning observations, leading to semantically incompatible matches and incoherent long-horizon plans. Still, it is interesting to visualize the resulting behavior: the robot produces locally coherent action chunks, but globally incoherent, see for instance otcfm_id_video.mp4.

Straightness & Energy We track the evolution during training of the straightness of the paths generated by the models (Fig. 2) and the energy (Fig. 3). For details on how we compute them, see Appendix C.1. Intuitively, paths with straightness closer to 1 are straighter, and paths with lower energies are smoother and more efficient, involving smaller velocities. Note that all methods converge to a constant average straightness and energy. The straightness of the CFM models is almost 1 while the straightness of DP remains lower, i.e., CFM models efficiently learn straight paths while DP learns curved and non-straight paths, which is consistent with the way they are trained. On the other hand, DP minimizes its energy and converges to a constant mean energy; note that the convergence coincides with the convergence of the straightness. On the other hand, the CFM models start with low energy and increase it up to a convergence mean value. Note also that the mean energy of the CFM models is lower than for DP, which is in alignment with the idea that CFM models (and especially FM) use an optimal transport map from noise to data samples, and thus achieve the minimal energy. In summary, *CFM-based policies maximize the straightness and minimize the energy of the denoising trajectory w.r.t. DP.*

Inference steps Additionally, because inference speed is critical in robotics, we also evaluate reduced integration steps. Using the best checkpoint of each model, we test different numbers of integration steps; see the in-distribution (ID) section in Table 2, where we have highlighted in bold the best-performing model for each number of integration steps (1, 10, and 100). For a more detailed visualization see Figure 11. Remarkably, with a single step, DP collapses to 0% success, while CFM models achieve strong performances, with FM-π_0 reaching 85% success. Thus, CFM-based models support effective one-shot action generation. This correlates with straightness results in Figure 4: CFM paths are nearly linear (the curves for the four CFM models are overlapped at $y \simeq 1$), whereas DP follows more non-linear trajectories, making single-step predictions inaccurate. We can visualize the stark difference in 1-shot robot behavior for both approaches in Figure 7: 1-shot DP actions follow random movements (second row in Fig. 7), while 1-shot actions generated by CFM variants are smooth and successfully complete the task (see first row in Fig. 7 for FM). For the full animations, see dp_id_video.mp4 for DP and cfm_id_video.mp4 for FM. In other words,

Table 2: Performance across ID and OOD settings with varying number of inference steps.

	ID			OOD		
Method	1-shot	10-shot	50-shot	1-shot	10-shot	50-shot
FM	0.23	0.88	0.95	0.46	0.67	0.80
RF	0.61	0.94	**0.98**	0.41	0.67	0.77
FM-π_0	**0.85**	0.94	0.95	**0.51**	0.64	0.67
RF-π_0	0.60	0.91	0.94	0.45	**0.74**	0.66
OT-CFM	0.0	0.0	0.0	0.0	0.0	0.0
DP	0.0	**0.99**	0.97	0.0	**0.74**	**0.92**

unlike DP, CFM-based policies are one shot action generators.

Finally, inference times across models are nearly identical (Figure 12), with a small but consistent speedup for CFM-based methods at higher integration counts, possibly due to small numerical differences between the Euler-based integration in CFM models and the denoising process in DP.

Generalization to an OOD Task We next evaluate generalization to a more challenging out-of-distribution (OOD) setting by adding a third block that obstructs the direct path between the cubes, requiring the robot to take a detour. To study OOD scaling, we train three versions of each model: using only the 200 in-distribution demonstrations (i.e., without obstacle), and adding either 20 or 40 demonstrations with the obstacle. Figure 13 shows the learning curves for FM, RF, and DP (the π_0 variants are omitted for clarity, as their trends closely match FM and RF). DP slightly outperforms the CFM models, but all methods exhibit similar scaling as more OOD data is provided; see Fig. 6. Evaluating the best checkpoints on 100 OOD tests yields the same pattern observed in-distribution: with 1-step inference, CFM-based policies overperform again DP, see the OOD section in Table 2. We can see the difference in 1-shot robot behavior for both approaches in Figure 7: 1-shot DP actions follow random movements (fourth row in Fig. 7), while 1-shot actions generated by CFM variants are smooth and successfully complete the task (see third row in Fig. 7 for FM). For the full animations, see dp_ood_video.mp4 for DP and cfm_ood_video.mp4 for FM. Hence, *CFM-based policies also exhibit strong single-step generalization, whereas DP fails under single-step inference.* With two integration steps DP matches the CFM models, and for larger step counts all five methods perform similarly, with DP showing a slight overall advantage. For more values of the integration step, see Table 2 and Fig. 14.

4 Discussion

Our results provide empirical evidence that the geometric structure of generative models has a direct impact on sampling efficiency. On CIFAR-10, we find that enforcing straighter trajectories via OT-CFM reduces the kinetic energy of the flow but does not improve FID over Rectified Flows, suggesting that while OT provides a theoretically optimal target, the robust straight-line conditional paths learned by RF are already sufficient to achieve state-of-the-art performance. In the robotics setting, however, we observe a much tighter link between geometry and control: the "straightness" of the learned flow correlates strongly with 1-shot success, and distilling policies into rectified flows

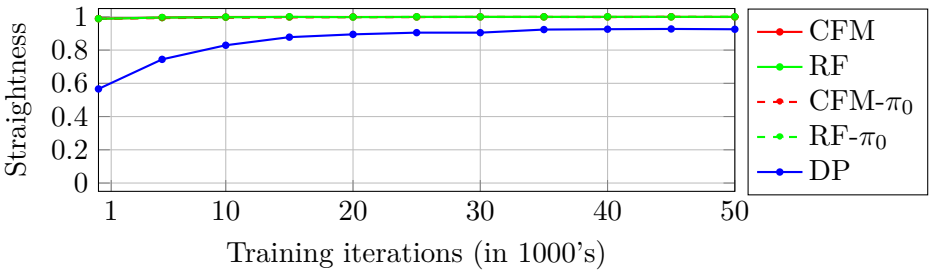

Figure 2: Evolution of the straightness of the integration path throughout training (using 100 integration steps for all models). All four CFM-based models are overlapped at $y \simeq 1$.

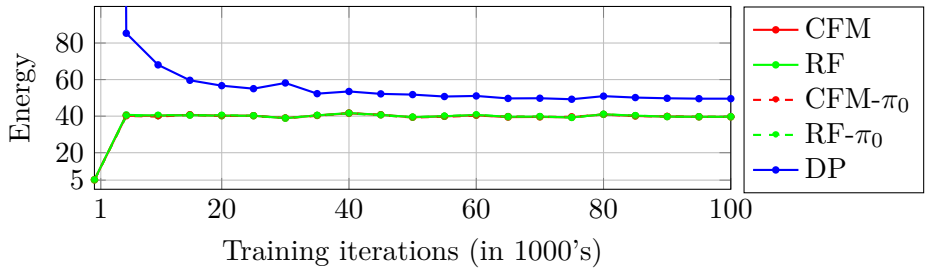

Figure 3: Evolution of the energy of the integration path throughout training (using 100 integration steps for all models). For the first iteration, the energy for DP is 3631.8.

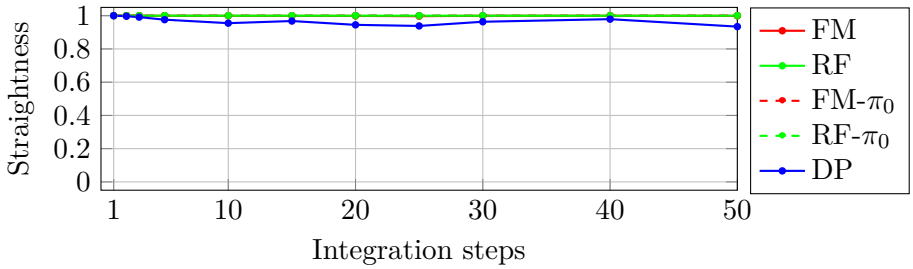

Figure 4: Straightness of the path vs. integration steps in the stack-cube task.

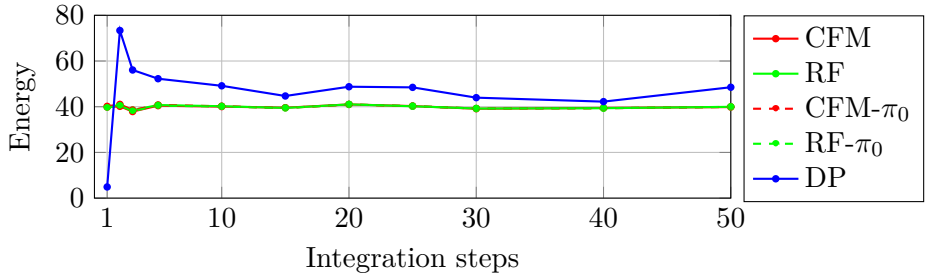

Figure 5: Energy of the generated paths vs. integration steps.

enables substantial step-size coarsening without catastrophic failurea robustness property that standard Diffusion Policy does not exhibit.

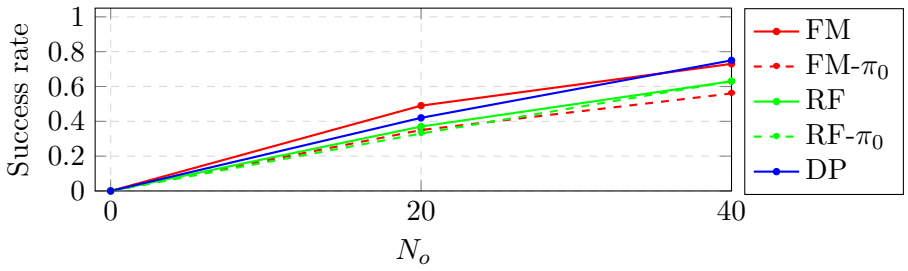

Figure 6: Success rate in the OOD task vs. number of OOD demonstrations showed during training.

5 Conclusion & Future Work

In this work, we disentangled the benefits of Optimal Transport alignment in continuous-time generative models. We found that while global OT-consistency (OT-CFM) yields the most geometrically efficient paths, it does not outperform simpler straight-line conditional paths (Rectified Flow) in terms of sample quality (FID). However, the geometric properties of Flow Matching models prove critical in low-latency robotic control. In the action generation domain, unlike Diffusion Policies, which require iterative refinement to produce usable actions, Flow Matching and Rectified Flow policies act as effective one-shot action generators, achieving high success rates with a single function evaluation. Future work should investigate larger batch sizes for OT-CFM to reduce coupling noise and explore the application of Rectified Flows to higher-frequency control loops where the 1-shot capability can be fully exploited for real-time reactivity on hardware.

Note on AI Use

During this project, we used large language models (GPT-5 Pro, GPT-5, and Gemini 3) to help improve the writing, debug code, and generate boilerplate code snippets.

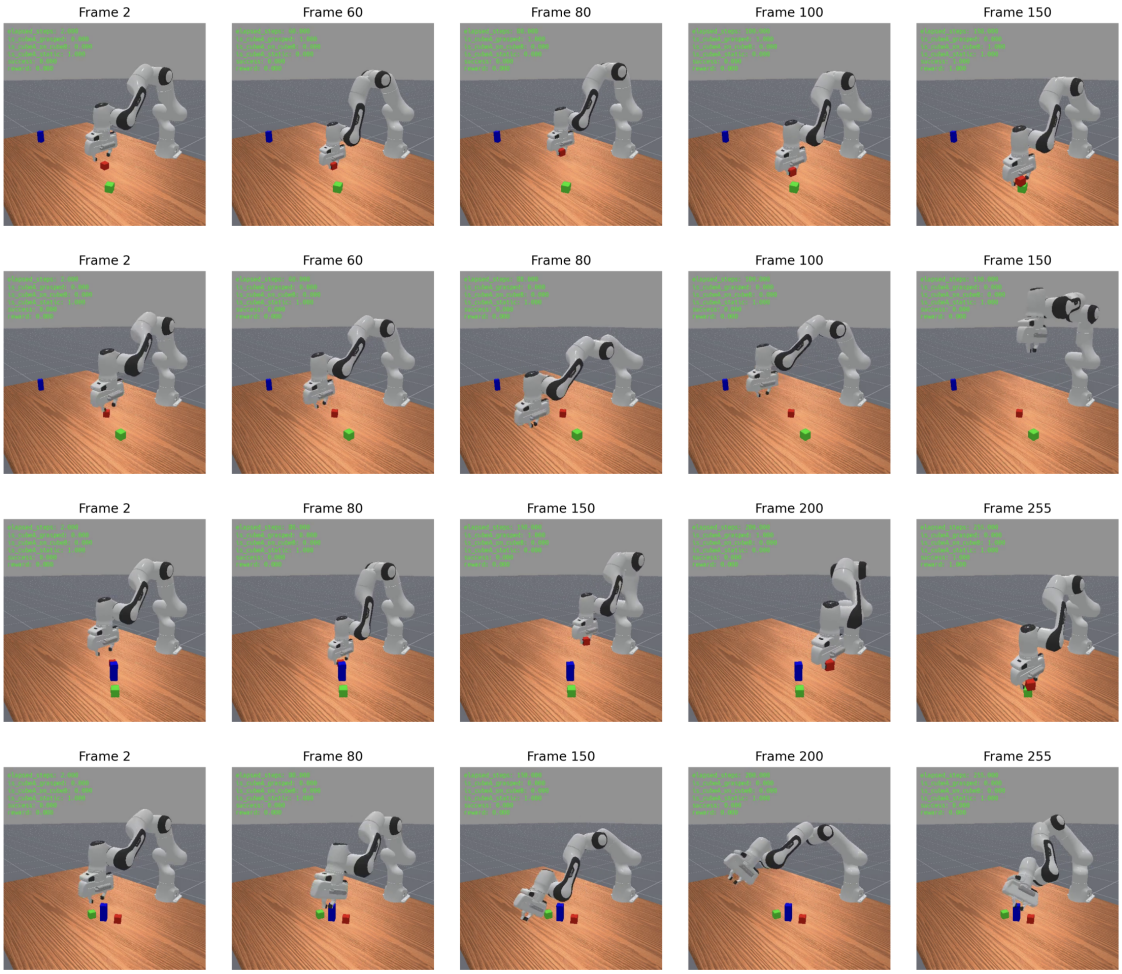

Figure 7: Comparison of FM vs DP with 1-shot inference on in-distribution (ID) and out-of-distribution (OOD) settings. From top row to bottom row: (i) FM in-distribution, (ii) DP in-distribution, (iii) FM OOD, (iv) DP OOD.

References

[1] Michael S. Albergo, Nicholas M. Boffi, and Eric Vanden-Eijnden. Stochastic Interpolants: A Unifying Framework for Flows and Diffusions. 2023. doi: 10.48550/ARXIV.2303.08797. URL https://arxiv.org/abs/2303.08797.

[2] Jascha Sohl-Dickstein, Eric A. Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics, 2015. URL https://arxiv.org/abs/1503.03585.

[3] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.

[4] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-Based Generative Modeling through Stochastic Differential Equations. 2020. doi: 10.48550/ARXIV.2011.13456. URL https://arxiv.org/abs/2011.13456.

[5] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.

[6] Gabriel Peyré and Marco Cuturi. Computational Optimal Transport. *arXiv*, 2018. doi: 10.48550/ARXIV.1803.00567. URL https://arxiv.org/abs/1803.00567.

[7] Jean-David Benamou and Yann Brenier. A computational fluid mechanics solution to the Monge-Kantorovich mass transfer problem. *Numerische Mathematik*, 84(3):375–393, jan 1 2000. ISSN 0029-599X. doi: 10.1007/s002110050002. URL http://dx.doi.org/10.1007/s002110050002.

[8] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. *arXiv preprint arXiv:2209.03003*, 2022.

[9] Qiang Liu. Rectified flow: A marginal preserving approach to optimal transport. *arXiv preprint arXiv:2209.14577*, 2022.

[10] Alexander Tong, Kilian Fatras, Nikolay Malkin, Guillaume Huguet, Yanlei Zhang, Jarrid Rector-Brooks, Guy Wolf, and Yoshua Bengio. Improving and generalizing flow-based generative models with minibatch optimal transport, 2024. URL https://arxiv.org/abs/2302.00482.

[11] Aram-Alexandre Pooladian, Heli Ben-Hamu, Carles Domingo-Enrich, Brandon Amos, Yaron Lipman, and Ricky T. Q. Chen. Multisample flow matching: Straightening flows with minibatch couplings. In Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett, editors, *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 28100–28127. PMLR, 23–29 Jul 2023. URL https://proceedings.mlr.press/v202/pooladian23a.html.

[12] Robert J McCann. A convexity principle for interacting gases. *Advances in mathematics*, 128 (1):153–179, 1997.

[13] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009.

[14] Fanbo Xiang, Yuzhe Qin, Kaichun Mo, Yikuan Xia, Hao Zhu, Fangchen Liu, Minghua Liu, Hanxiao Jiang, Yifu Yuan, He Wang, Li Yi, Angel X. Chang, Leonidas J. Guibas, and Hao Su. SAPIEN: A simulated part-based interactive environment. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020.

[15] Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, page 02783649241273668, 2023.

[16] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018. URL https://proceedings.neurips.cc/paper_files/paper/2018/file/69386f6bb1dfed68692a24c8686939b9-Paper.pdf.

[17] Will Grathwohl, Ricky T. Q. Chen, Jesse Bettencourt, Ilya Sutskever, and David Duvenaud. Ffjord: Free-form Continuous Dynamics for Scalable Reversible Generative Models. 2018. doi: 10.48550/ARXIV.1810.01367. URL https://arxiv.org/abs/1810.01367.

[18] *Solving Ordinary Differential Equations I.* Springer Berlin Heidelberg, 1993. ISBN 9783540566700. doi: 10.1007/978-3-540-78862-1. URL http://dx.doi.org/10.1007/978-3-540-78862-1.

[19] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[20] Stone Tao, Fanbo Xiang, Arth Shukla, Yuzhe Qin, Xander Hinrichsen, Xiaodi Yuan, Chen Bao, Xinsong Lin, Yulin Liu, Tse kai Chan, Yuan Gao, Xuanlin Li, Tongzhou Mu, Nan Xiao, Arnav Gurha, Viswesh Nagaswamy Rajesh, Yong Woo Choi, Yen-Ru Chen, Zhiao Huang, Roberto Calandra, Rui Chen, Shan Luo, and Hao Su. Maniskill3: Gpu parallelized robotics simulation and rendering for generalizable embodied ai. *Robotics: Science and Systems*, 2025.

Supplementary Information

A Reproducibility

For image generation, all code for our experiments (model training and evaluations) can be found on our image-generation github repo. For action generation, all code for our experiments (model training and evaluations) can be found on our robot action-generation github repo.

Training We train a CIFAR-10 generative model based on a UNet architecture. The network operates on 32×32 RGB images and is implemented as a UNet with a base channel width of 128, two residual blocks per resolution level, and channel multipliers [1, 2, 2, 2], yielding a multi-scale encoderdecoder with skip connections. Self-attention is applied at the 16×16 spatial resolution with 4 attention heads and 64 channels per head, and a dropout rate of 0.1 provides regularization inside the UNet. We train on the CIFAR-10 training split, using random horizontal flips as augmentation and normalizing each color channel to fit approximately in $[-1, 1]$.

For the non-diffusion models, we use Adam optimization with a target learning rate of 2×10^{-4}, applied to mini-batches of size 128 for a total of 300000 update steps. The learning rate follows a simple linear warm-up schedule, increasing linearly from 0 to the target value over the first 5000 steps and then held constant for the remainder of training. To stabilize training the total gradient norm is clipped to 1 at every step, and an exponential moving average of the model parameters with decay factor 0.9999 is maintained and used for evaluation and sample generation.

The diffusion model specifically is defined by a linear beta schedule with 1000 time steps, where the noise variance β_t increases linearly from 10^{-4} to 0.02; from these, the code computes the usual DDPM quantities to implement both the forward corruption process $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\, \varepsilon$ and the reverse-time sampling dynamics. During training, for each mini-batch of 256 images the model samples a random timestep $t \in \{0, \ldots, T-1\}$, constructs a noisy image x_t together with the exact Gaussian noise ε used to corrupt the clean image x_0, and is trained to predict this noise from (t, x_t). The loss was the MSE between the predicted and true noise, averaged over images and timesteps. Here we optimize using Adam with a target learning rate of 2×10^{-4} and gradient norm clipping at 1.0 to stabilize updates, trained for 200000 optimization steps. The learning rate follows a two-stage schedule: it is linearly warmed up from 0 to the base value over the first 5000 steps, then we use cosine decay to reduce to 10% of the effective learning rate by the end of training. An exponential moving average of the model parameters with decay 0.9999 is maintained throughout training.

Model training was done on Kempner H100 GPUs.

Evaluation We evaluate generative quality with the Fréchet Inception Distance (FID) using the `clean-fid` implementation. For each solver configuration, the model generates 10000 CIFAR10resolution images, which are evaluated against the CIFAR10 test set statistics using the default `clean-fit` Inception feature space. All FID evaluations use a batch size of 64 unless otherwise specified.

Given a trained model checkpoint, our evaluation script runs a set of predefined solver configurations, using fixed-step Euler, midpoint, and `RK4` schemes, as well as adaptive solvers such as `Dopri5`. This allows us to compare the effect of numerical methods used to integrate the models learned

continuous-time dynamics. Fixed-step solvers differ mainly in order of accuracy and cost: Euler is simple but low-accuracy; midpoint improves stability with moderate extra cost; `RK4` provides high accuracy but requires four evaluations per step. Adaptive solvers automatically adjust step sizes based on local error estimates, taking smaller steps in regions where the vector field changes rapidly and larger steps where it is smooth. These differences directly affect the geometric character of sample trajectories.

To evaluate solver efficiency and numerical behavior, each configuration is additionally run on a batch of 128 noise samples to compute (1) the number of function evaluations (NFE) and (2) a trajectory straightness metric (this is discussed in later paragraphs). Solver settings follow a fixed grid: Euler, midpoint, and RK4 solvers are run with 5, 10, 20, 50, and 100 uniform steps over the interval [0,1], with NFEs corresponding to their respective numbers of internal evaluations (e.g., 1, 2, and 4 evaluations per step). Adaptive experiments use `Dopri5` with a range of tolerance values $(10^{-3}, 10^{-4}, 10^{-5})$, allowing the integrator to choose its own step sequence.

We also quantify the geometric straightness of the generated sample paths. Given a trajectory tensor shape $[T, B, C, H, W]$ (time steps, batch, channels, height, width), we first flatten the spatial dimensions to obtain $[T, B, D]$ with $D = CHW$. For each sample in the batch, we compute the direct distance (euclidean norm) between the initial and final states, and the path length as the sum of step-wise euclidean distances across all recorded time points. The straightness score for each trajectory is defined as

$$\text{straightness} = \frac{\text{direct distance}}{\text{path length} + 10^{-8}}$$

and we report the average value of this over the batches as our final metric. For flow models, these trajectories are generated either with a fixed-step ODE solver (Euler, midpoint, or `RK4`)) on a uniform grid, or with an adaptive solver where we evaluate straightness on a coarse, fixed grid of 50 evaluation times. For the diffusion model (DDPM), we build trajectories by sampling 50 intermediate states along the reverse chain using DDPM update.

Kinetic energy is measured differently for continuous-time flow models and discrete-time diffusion models, but in both cases is intended to approximate an action-like quantity along the generative path. For flow models, we work with the learned velocity field $v(t, x)$, augmenting the state from x to the pair (x, E). We then define ODEs

$$\frac{d}{dt}x = v(t, x)$$

and

$$\frac{d}{dt}E = \|v(t, x)\|^2$$

Integrating this augmented system with the same solver and tolerance as above yields a trajectory of energies, which we average over random initial conditions to get the kinetic energy metric.

For diffusion models, where we do not have a continuous velocity field, we instead use a BenamouBrenierstyle discrete kinetic energy: after flattening the diffusion trajectory to shape $[T, B, D]$, we compute per-step increments $\Delta x_t = x_{t+1} - x_t$, accumulate squared norms, and sum across time to obtain an energy per sample, which we again average over the batches.

B CIFAR-10

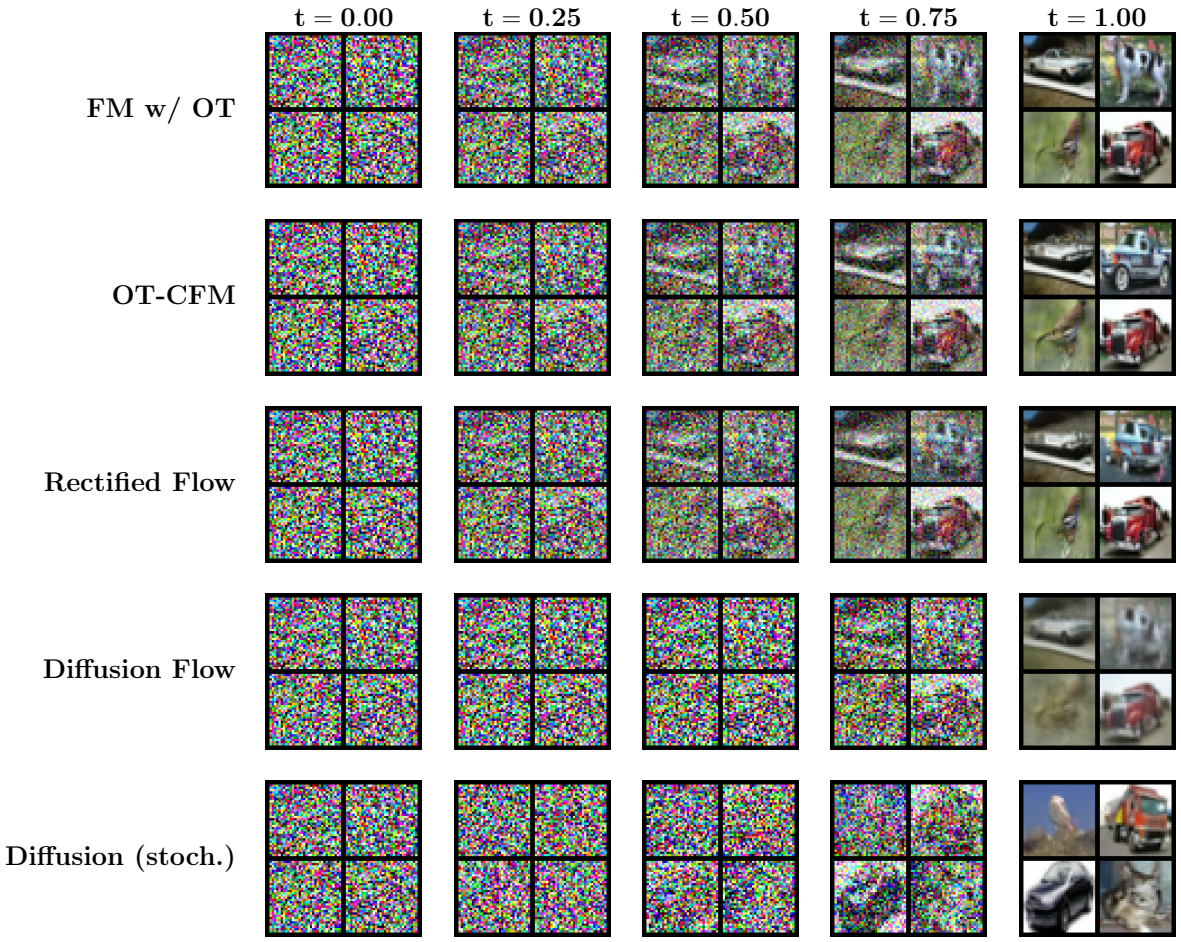

Figure 8: Evolution of samples along the generative trajectories of different models. Columns correspond to increasing normalized time t, from pure noise ($t = 0$) to final generated samples ($t = 1$). Flow-based models integrate the learned forward ODE; the Diffusion Flow baseline uses the deterministic induced flow; and Diffusion (stoch.) shows the standard noisy DDPM reverse process. All rows share the same initial noise for direct comparability.

C Generative modeling for robotics

We study how different generative modeling techniques affect robot learning performance. This setting allows a clear comparison between the training dynamics and integration-step sensitivity of diffusion and CFM models. Interest in flow-based controllers has grown recently, especially following Physical Intelligence (π_0) [19], whose Vision-Language-Action systems use a flow-matching controller. However, to the best of our knowledge, no open-source robotics implementation of CFM currently exists, even in widely used simulators such as ManiSkill[1], where such functionality has been requested[2], and no systematic comparison between different CFM models (including FM, RF and OT-CFM) and the state-of-the-art imitation learning method Diffusion Policy [15] has been made. Motivated by this gap, we investigate CFM for robotic control within the Diffusion Policy framework. We implement and compare Flow Matching [5], Rectified Flows [8, 10], OT-CFM [10], and standard diffusion policy [15] under the same architecture and hyperparameters. We first review Diffusion Policy and then show how CFM can serve as a drop-in alternative.

Diffusion Policy Diffusion Policies [15] formulate visuomotor control as a generative modeling problem using Denoising Diffusion Probabilistic Models (DDPMs) [3]. A DDPM generates samples by iteratively denoising a Gaussian noise vector $\mathbf{A}^K \sim \mathcal{N}(0, I)$ through a sequence $\mathbf{A}^K \to \mathbf{A}^{K-1} \to \cdots \to \mathbf{A}^0$, where $\mathbf{A}^0$ is the final action sequence. Each step removes a small amount of noise using a learned noise prediction network ε_θ. The reverse update has the generic form

$$\mathbf{A}_t^{k-1} = \alpha\Big(\mathbf{A}_t^k - \gamma\, \varepsilon_\theta(\mathbf{O}_t, \mathbf{A}_t^k, k)\Big) + \mathcal{N}(0, \sigma^2 I).$$

Training simply corrupts real data (expert demonstrations coming from, e.g., teleoperation or motion planners in simulation) by adding Gaussian noise $\varepsilon_k \sim \mathcal{N}(0, \sigma_k^2 I)$ and asks the model ε_θ to predict the added noise using the MSE loss $\mathcal{L} = \mathrm{MSE}(\varepsilon_\theta(\mathbf{O}_t,\ \mathbf{A}_t^0 + \varepsilon_k,\ k), \varepsilon_k)$. Note that $\mathbf{A}_t^k$ is the (noisy) sequence of future robot actions, and $\mathbf{O}_t$ the current observations. At inference time, the policy receives the most recent observation window of length T_o and predicts a future action sequence of length T_p; only the first $T_a < T_p$ actions are executed before replanning. Following [15], we use T_o=2, T_p=16, and T_a=8.

Note that the ground-truth actions come from expert demonstrations, obtained either from real teleoperated robot trajectories or from actions generated in simulation (e.g., using motion-planning algorithms). In our case, we build two motion planners with ManiSkill that successfully perform the manipulation tasks: one for the standard stack-cube task, and a second for the stack-cube task with an obstacle present. Before training, each demonstration is segmented into shorter action sequences $\mathbf{A}_t$ of fixed length (e.g., 16 actions), together with the corresponding conditioning observations $\mathbf{O}_t$. For instance, 200 demonstrations of 150 actions each yield approximately $200 \times 150 = 30,000$ training samples (using sliding windows with stride 1.)

Finally, we remark on the definition of "actions." In Diffusion Policy, actions may represent either end-effector poses (e.g., a 6D pose parameterization) or the robots joint angles. Both choices typically work well for single-arm manipulation; in our experiments we use end-effector poses as the action representation.

[1] https://github.com/haosulab/ManiSkill

[2] https://github.com/haosulab/ManiSkill/issues/939

CFM based Policies An alternative to DDPM denoising is to learn the velocity field that transports noise to actions, following the Conditional Flow Matching (CFM) formalism. In this case, the loss becomes the CFM loss $\mathcal{L}^\tau(\theta) = \text{MSE}(v_\theta(\mathbf{A}_t^\tau, \mathbf{O}_t),\ u(\mathbf{A}_t^\tau \mid \mathbf{A}_t))\,,\ \tau \in [0,1]$, where t denotes robot timesteps and τ denotes flow-matching timesteps. Analogously to the diffusion modeling approach, given an initial noise sample $\mathbf{A}_t^0 \sim \mathcal{N}(0, I)$ and the target true action sequence $\mathbf{A}_t^1 = \mathbf{A}_t$, the model is trained by sampling noise $\varepsilon \sim \mathcal{N}(0, I)$, constructing the noisy actions $\mathbf{A}_t^\tau = \mu_\tau + \sigma_\tau \epsilon$ from the corresponding probability path $p_\tau(x \mid \mathbf{A}_t)$ and training the model outputs $v_\theta(\mathbf{A}_t^\tau, \mathbf{O}_t)$ to match the target conditional vector field $u(\mathbf{A}_t^\tau \mid \mathbf{A}_t)$. At inference time, actions are generated by integrating the learned vector field from $\tau = 0$ to $\tau = 1$, starting from random noise $A_t^0 \sim \mathcal{N}(0, I)$. In general the forward Euler integration scheme is used:

$$A_t^{\tau+\delta} = A_t^\tau + \delta \cdot v_\theta(A_t^\tau, o_t)\,,$$

where δ denotes the integration step size. Following this methodology, FM, OT-CFM, or RF can be used, the only difference being the choice of probability path p_t connecting the source and target distributions, as well as the way the conditional vector field u is computed (e.g., independent coupling, minibatch OT coupling).

C.1 Straightness and Energy

We now describe how straightness and energy are computed for CFM-based policies and for Diffusion Policy. Because the two models rely on different generative processes, the quantities must be evaluated using different procedures. Nevertheless, as we show below, the underlying definitions are equivalent: in both cases, the metrics characterize equivalent properties of the trajectory taken in latent action space.

Note that we use DDIM rather than DDPM for measuring straightness and energy because DDIM produces a deterministic denoising trajectory. This ensures that the path in latent action space is well defined and reproducible, which is essential for computing geometric quantities such as path length and integrated energy. In contrast, DDPM sampling introduces stochastic noise at every step, making the trajectory random and thus unsuitable for consistent geometric evaluation.

Straightness (CFMs) During forward integration, the CFM model begins from an initial latent action a_0 and integrates a learned velocity field $v_\theta(a_t, t)$ over time $t \in [0, 1]$. Let a_1 denote the final latent action after all forward-model steps. At each step, the model records the infinitesimal displacement $\Delta a_t = v_\theta(a_t, t)\,\Delta t$ and accumulates the total path length

$$L = \sum_t \|\Delta a_t\|_2.$$

The straight-line distance between the start and end of the trajectory is

$$D = \|a_1 - a_0\|_2.$$

The *straightness* metric is defined as the ratio

$$\text{Straightness} = \frac{D}{L + \varepsilon},$$

which equals 1 when the trajectory is perfectly straight and decreases as the trajectory becomes more curved.

Energy (CFMs) At each step, the velocity field predicts a velocity $v_t = v_\theta(a_t, t)$. The instantaneous kinetic energy is proportional to $\|v_t\|_2^2$. The model integrates this quantity over time to obtain the total energy

$$E = \frac{1}{2} \sum_t \|v_t\|_2^2 \, \Delta t,$$

or, when using Heun's method, replacing v_t by the average velocity $\frac{1}{2}(v_t + v_t^{\text{Euler}})$. This energy measures how "effortful" the forward-model rollout is: lower energy indicates smoother, more efficient trajectories.

Straightness (DP) Diffusion Policy generates actions by starting from Gaussian noise x_0 and iteratively denoising it through a deterministic DDIM sampler. Let x_k denote the latent action sequence after the k-th denoising step, and let K be the total number of diffusion iterations, with uniform step size $\Delta t = 1/K$. At each step, the displacement in latent action space is

$$\Delta x_k = x_{k+1} - x_k,$$

and the model accumulates the total path length

$$L = \sum_{k=0}^{K-1} \|\Delta x_k\|_2.$$

The straight-line distance between the initial noise x_0 and final denoised trajectory x_K is

$$D = \|x_K - x_0\|_2.$$

The straightness metric is therefore

$$\text{Straightness} = \frac{D}{L + \varepsilon},$$

which quantifies how directly the denoising trajectory moves through latent action space.

Energy (DP) During DDIM denoising, each step induces an effective "velocity" in latent space,

$$v_k = \frac{\Delta x_k}{\Delta t},$$

from which the model accumulates the total energy as

$$E = \frac{1}{2} \sum_{k=0}^{K-1} \|v_k\|_2^2 \, \Delta t.$$

This corresponds to the time-integral of latent-space kinetic energy along the DDIM denoising trajectory. Again, lower energy indicates that the denoising path is smoother and requires less "effort" to reach the final action sequence.

C.2 Additional Figures

Note that we use

$$p(\tau) = \text{Beta}\left(\frac{s-\tau}{s};\, 1.5,\, 1\right) = \frac{3}{2s}\left(\frac{s-\tau}{s}\right)^{1/2}, \quad \tau \in [0, s].$$

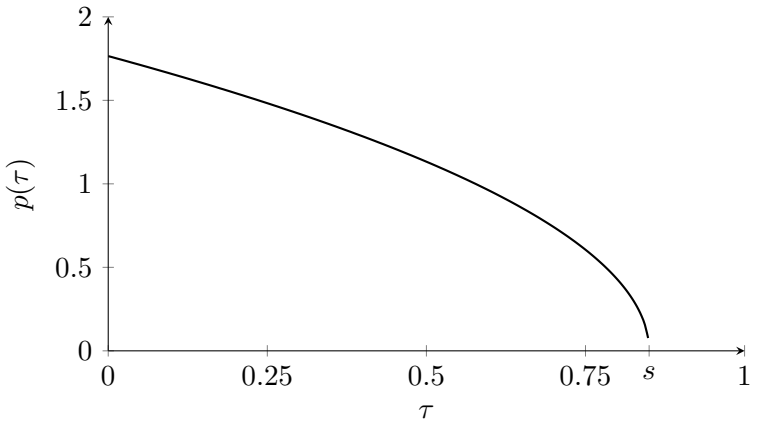

Figure 9: Timestep sampling distribution with $p(\tau) = \text{Beta}\left(\frac{s-\tau}{s}; 1.5, 1\right)$ with $s = 0.85$.

C.2.1 ID Results

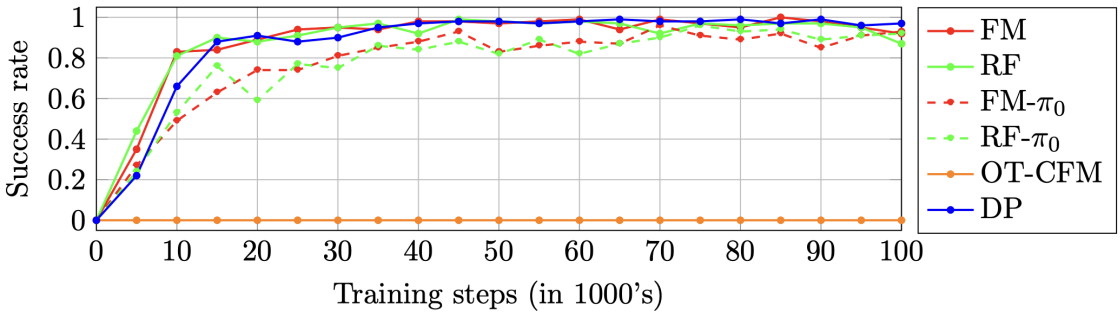

Figure 10: Evaluation results for the five models during training for the stack-cube task. Success rates are averaged over 100 tests, and measured each 5,000 training steps.

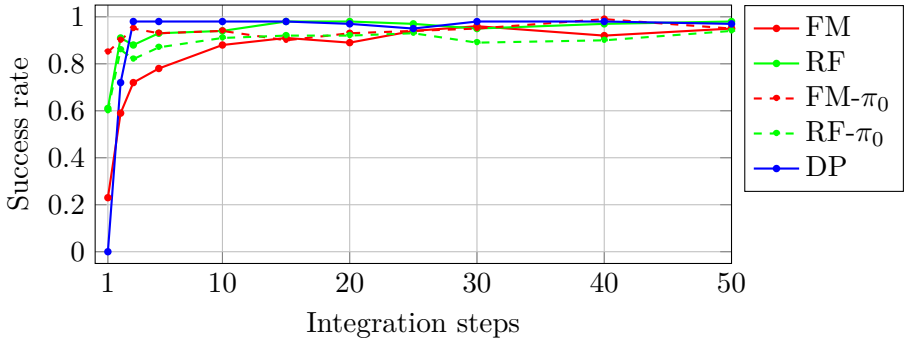

Figure 11: Success rate vs. integration steps in the stack-cube task.

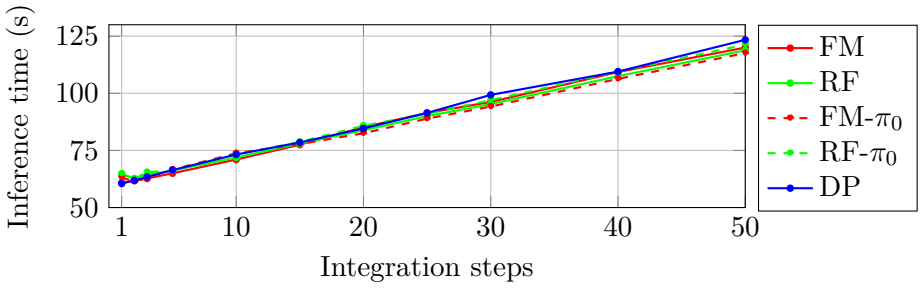

Figure 12: Running time vs. integration steps for 100 evaluation tests in the stack-cube task.

C.2.2 OOD Results

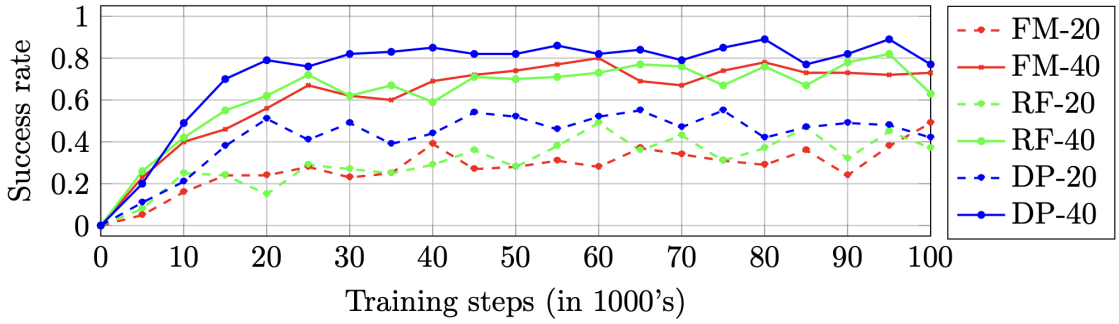

Figure 13: Evaluation results for three different models during training for the OOD stack-cube task. Success rates are averaged over 100 tests, and measured each 5,000 training steps.

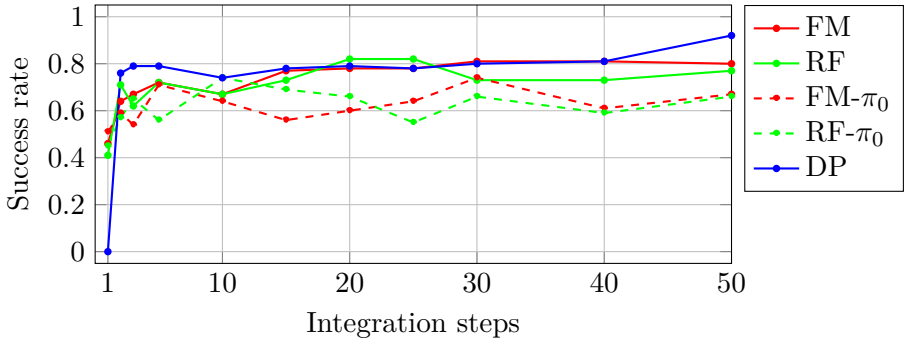

Figure 14: Success rate vs. integration steps for the stack-cube OOD test (presence of an obstacle).